

Week 13: K-Nearest Neighbors (KNN) & Decision Tree (DT) Classification

Submission Guidelines:

1. Each task must properly show the output. DO NOT Clear the outputs after running.
2. **Download the .ipynb file** from Jupyter Notebook.
3. **Upload the .ipynb** to GCR.
4. **Failure to follow the submission guidelines will result in a deduction of 50% of the obtained marks.**

Task 1: K-Nearest Neighbors (KNN) Classification with Hyperparameter Optimization and Decision Boundary Analysis

Objective

To implement and critically analyze the K-Nearest Neighbors (KNN) algorithm for supervised classification tasks, with emphasis on distance metric selection, optimal hyperparameter tuning (K-value), and visualization of decision boundaries in a multi-dimensional feature space.

Dataset Specification

You are required to utilize the **Iris Dataset** (UCI Machine Learning Repository / Scikit-learn built-in dataset), consisting of:

- **Instances:** 150 samples
- **Features:** 4 continuous attributes
 - Sepal length
 - Sepal width
 - Petal length
 - Petal width
- **Classes:** 3 species
 - Setosa
 - Versicolor
 - Virginica

Dataset must be loaded via `sklearn.datasets.load_iris()` or equivalent preprocessing pipeline.

Implementation Requirements

Students must implement KNN using **Scikit-learn library**, ensuring:

1. Data preprocessing:

- Feature normalization using **StandardScaler** or **MinMaxScaler**
- Handling of feature scaling sensitivity due to distance-based learning

2. Model training:

- Implementation of KNN classifier using `KNeighborsClassifier`
- Evaluation of multiple K values (e.g., K = 1 to 20)

3. Distance metrics exploration:

- Euclidean distance (mandatory)
- Manhattan distance (optional extension)

4. Train-test split:

- 80/20 or 70/30 stratified splitting
- Fixed random state for reproducibility

Evaluation Metrics

Students must compute and interpret:

- Accuracy Score
- Precision, Recall, F1-score (macro and weighted averages)
- Confusion Matrix

Required Visualizations

The following graphical analyses must be included:

1. K vs Accuracy Plot

- Line graph depicting variation in classification accuracy with respect to different K values

2. Confusion Matrix Heatmap

- Visual representation of classification correctness and misclassification distribution

3. Decision Boundary Visualization

- Color-coded class regions showing KNN partitioning behavior

4. Data Distribution Plot

- Pairplot or scatter matrix to analyze class separability

Task 2: Decision Tree Classification with Entropy/Gini Analysis and Tree Pruning Optimization

Objective

To construct, evaluate, and optimize a Decision Tree classifier by analyzing impurity measures (Gini Index and Entropy), controlling overfitting through pruning strategies, and interpreting hierarchical decision structures.

Dataset Specification

Students must utilize the **Breast Cancer Wisconsin Dataset** (Scikit-learn built-in dataset), comprising:

- **Instances:** 569 samples
- **Features:** 30 numerical attributes (computed from digitized images of breast mass)
- **Classes:**
 - Malignant
 - Benign

Dataset loading via `sklearn.datasets.load_breast_cancer()` is recommended.

Implementation Requirements

Students are required to implement Decision Tree classification using **Scikit-learn DecisionTreeClassifier**, ensuring:

1. Data preprocessing:

- Feature scaling (optional but recommended for consistency in comparison)
- Train-test split (80/20 stratified)

2. Model construction:

- Training using both impurity criteria:
 - Gini Index
 - Entropy (Information Gain)

3. Hyperparameter tuning:

- Maximum depth (`max_depth`)

- Minimum samples split (min_samples_split)
- Minimum samples leaf (min_samples_leaf)

4. Overfitting control:

- Pre-pruning techniques (parameter constraints)
- Post-pruning via cost complexity pruning (ccp_alpha)

Evaluation Metrics

Students must report:

- Classification Accuracy
- Precision, Recall, F1-score
- Confusion Matrix
- ROC-AUC Curve (mandatory for binary classification)

Required Visualizations

The following analytical and interpretive plots are mandatory:

1. **Decision Tree Structure Visualization**
 - Graphical representation of trained tree using plot_tree or Graphviz
 - Node-level interpretation of splits and conditions
2. **Feature Importance Plot**
 - Ranking of attributes based on contribution to classification
3. **ROC Curve**
 - True Positive Rate vs False Positive Rate comparison for model evaluation
4. **Impurity Comparison Plot**
 - Gini vs Entropy performance comparison across model configurations
5. **Depth vs Accuracy Curve**
 - Visualization of overfitting/underfitting trade-off

Task 3: Image Classification using KNN

Problem Statement

Apply KNN for handwritten digit recognition using the **Digits dataset**.

Dataset Specifications

- Images: 8×8 grayscale
- Classes: Digits 0–9

Required Steps

1. Load dataset using `load_digits()`
2. Flatten image matrix into feature vectors
3. Standardize features
4. Train KNN classifier
5. Predict test samples

Required Visualizations

1. Display 10 sample digit images with labels
2. Confusion Matrix Heatmap
3. Misclassified digit samples visualization

TASK 4: K-Nearest Neighbors (KNN) with KD-Tree Optimization

Objective

To implement the K-Nearest Neighbors (KNN) algorithm using a KD-Tree based spatial indexing structure and evaluate its computational efficiency against brute-force nearest neighbor search, particularly in terms of query time and scalability behavior.

Dataset Description

- Dataset: Wine Dataset (Scikit-learn)
- Instances: 178 samples
- Features: 13 continuous chemical attributes of wine
- Classes: 3 cultivars of wine

Implementation Requirements

Students are required to:

1. Load dataset using `sklearn.datasets.load_wine()`
2. Perform feature normalization using **StandardScaler**
3. Split dataset into training and testing sets (80/20 stratified split)

4. Train KNN classifier using:
 - `algorithm = 'kd_tree'`
 - `n_neighbors` optimized in range 1–20
5. Train a baseline KNN model using:
 - `algorithm = 'brute'`
6. Conduct comparative evaluation between KD-Tree and brute-force approaches in terms of:
 - prediction latency
 - classification performance consistency

Evaluation Metrics

- Accuracy Score
- Precision, Recall, F1-score
- Confusion Matrix
- Computational inference time comparison

Required Visualizations

1. KD-Tree vs Brute Force Execution Time Comparison (Bar Plot)
2. K vs Accuracy Curve
3. Confusion Matrix Heatmap
4. Pairwise Feature Distribution (Scatter Matrix / Pairplot)

Task 5: Hyperparameter Optimization of KNN via Cross Validation

Problem Statement

Implement a **K-Nearest Neighbors classifier** on the **Wine dataset** and determine the most suitable value of **K** using **5-Fold Cross Validation**. Analyze how neighborhood size influences predictive generalization.

Dataset Specifications

`load_wine()`

- Total Samples: 178
- Features: 13 continuous chemical attributes

- Classes: 3 wine categories

Required Implementation Steps

1. Load dataset and inspect dimensions.
2. Perform train-test split using: 80% training, 20% testing
3. Apply feature standardization.
4. Evaluate KNN for: $K = 1, 3, 5, 7, 9, 11, 13, 15$
5. Use: 5-Fold Cross Validation
6. Store mean validation accuracy for each K .
7. Identify optimal K .
8. Retrain KNN using best K .
9. Evaluate final model on test set.

Required Visualizations

1. Line graph: K vs Mean Cross Validation Accuracy
2. Confusion Matrix of final model
3. Bar chart of Precision, Recall, F1-score

Required Analytical Questions

1. Why is cross validation more reliable than a single train-test split?
2. Why are odd K values preferred?
3. Which K produced maximum validation performance?
4. What symptoms indicate overfitting at low K ?
5. What symptoms indicate underfitting at high K ?

TASK 6: Decision Tree Regression with Complexity Control and Pruning Analysis

Objective

To implement Decision Tree Regression and critically analyze the effect of structural complexity parameters and pruning strategies on predictive accuracy and generalization capability.

Dataset Description

- Dataset: California Housing Dataset (Scikit-learn)

- Instances: 20,640 samples
- Features: 8 socio-economic and geographic attributes
- Target Variable: Median house value (continuous output)

Implementation Requirements

Students must:

1. Load dataset using `sklearn.datasets.fetch_california_housing()`
2. Perform 80/20 train-test split
3. Implement `DecisionTreeRegressor`
4. Experiment with hyperparameters:
 - `max_depth`
 - `min_samples_split`
 - `min_samples_leaf`
5. Apply **cost complexity pruning** using `ccp_alpha`
6. Compare model behavior across different complexity levels

Evaluation Metrics

- Mean Absolute Error (MAE)
- Mean Squared Error (MSE)
- Root Mean Squared Error (RMSE)
- R^2 Score

Required Visualizations

1. Actual vs Predicted Scatter Plot
2. Tree Depth vs RMSE Curve
3. Feature Importance Ranking Plot
4. Residual Error Distribution Plot